Proyecto Final.

Realizar procesamiento de vector de 4 bandas mediante Knn.

Tema:
Algoritmo de clasificación (KNN)

Universidad:
Facultad de Informática de Culiacán (FIC).

Materia:
Tema Selecto e Inteligencia Artificial

Maestro/a:
Arturo Yee Rendon.

Grupo:
4-3.

Alumno::
Norberto Arredondo Vizcarra.
Jesus Randu Quintero Parra.

Fecha de Actualización:
01/06/2024

Instrucciones de desarrollo.

Utilizar los conocimientos obtenidos en las asignaturas de Inteligencia Artificial y Temas Selectos para crear un algoritmo que clasifique los píxeles de una imagen como tierra o agua.
Descripción de los archivos

• El archivo rsTraining.txt contiene 200 puntos, cada uno con cinco columnas. Las primeras cuatro columnas representan las propiedades de los píxeles, mientras que la última columna indica si el punto es tierra (0) o agua (1).
• Los archivos band1.irs, band2.irs, band3.irs y band4.irs contienen imágenes satelitales de Kolkata en cuatro bandas diferentes. Cada imagen tiene un tamaño de 512x512 píxeles. El uso combinado de estos archivos conforma la imagen completa de la ciudad de Kolkata.

Parte medular

Primeramente después de la lectura de los datos, lo realizado fue revolver dichos datos para que en cada ejecución se tomarán datos distintos y que estos no fueran los mismos para entrenar y explotar el modelo.

```
np.random.shuffle(Datos)
```

Una vez que teníamos revueltos los datos, lo siguiente fue la separación de dichos datos para que un porcentaje (75%) fueran para el entrenamiento y los datos restantes (25%) fueran para la explotación del modelo.

```
DatosEnt=[]
DatosExp=[]

for i in range (0,150):
    DatosEnt.append(Datos[i])

for i in range(150,200):
    DatosExp.append(Datos[i])
```

Una vez que ya teníamos los datos separados para el entrenamiento y la explotación, lo siguiente fue separar las características de los datos y su etiqueta para cada uno de ellos tanto de la sección de entrenamiento como para la de explotación.

```
DatoEnt_X=[]
DatoEnt_Y=[]
for i in DatosEnt:
    DatoEnt_Y.append(i[4])
    DatoEnt_X.append(i[0:4])

DatoExp_X=[]
DatoExp_Y=[]
for j in DatosExp:
    DatoExp_Y.append(j[4])
    DatoExp_X.append(j[0:4])
```

Una vez obtenidos todos los datos en las secciones correspondientes y necesarias para el modelo, lo siguiente fue la lectura de las bandas o imágenes satelitales bajo la dimensiones necesarias, que en este caso fueron de 512x512.

```
Imagen1=np.fromfile("band1.irs",dtype=np.int8)
Banda1=np.reshape(Imagen1,(512,512))

Imagen2=np.fromfile("band2.irs",dtype=np.int8)
Banda2=np.reshape(Imagen2,(512,512))

Imagen3=np.fromfile("band3.irs",dtype=np.int8)
Banda3=np.reshape(Imagen3,(512,512))

Imagen4=np.fromfile("band4.irs",dtype=np.int8)
Banda4=np.reshape(Imagen4,(512,512))
```

Una vez que tenemos la lectura de las bandas o imágenes satelitales lo siguiente fue crear un vector con cada punto de las bandas para generar una lista con las mismas dimensiones de las características de los datos.

```
Vector=[]
for i in range(0,512):
    for j in range(0,512):

Vector.append([float(Banda1[i][j]),float(Banda2[i][j]),float(Banda3[i][j]),float(
Banda4[i][j])])
```

Una vez recopilado y creado todos los datos necesarios, lo siguiente fue crear el modelo de KNN y pasarle los datos de entrenamiento y de explotación necesarios para el modelo y calcular el porcentaje de precisión para saber cual es el mejor número de K vecinos.

```
Modelo = KNeighborsClassifier(n_neighbors=3)
Modelo.fit(DatoEnt_X,DatoEnt_Y)
PredDatoExp_Y= Modelo.predict(DatoExp_X)
Precision= accuracy_score(DatoExp_Y,PredDatoExp_Y)
print("Precision = ", Precision)
```

Una vez entrenado el modelo y descubierto el mejor K vecino, lo siguiente fue pasarle los datos de las bandas o imágenes para hacer la clasificación correcta de estos datos y con todos los datos de clasificación en este caso y no con un porcentaje como en el modelo anterior.

```
MejorModelo = KNeighborsClassifier(n_neighbors=3)
MejorModelo.fit(DatosTotales_X,DatosTotales_Y)
PredDatos = MejorModelo.predict(VectorBandas)
```

Una vez que el modelo ya clasificó los datos de las bandas, lo siguiente fue crear una matriz con ellos y asignar le la misma cantidad de píxeles o dimensiones de las bandas para que posteriormente se pueda generar adecuadamente la imagen.

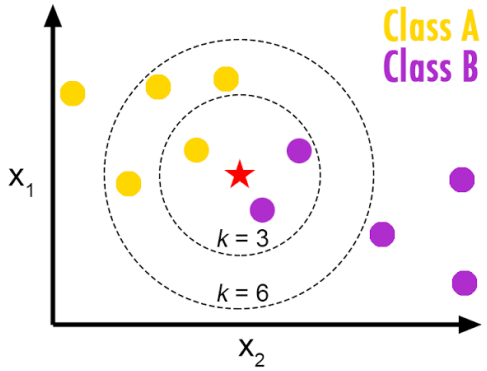
```
VectorImagen = np.matrix(PredDatos)

ImagenSalida = np.reshape(VectorImagen,(512,512))
```

Una vez teniendo las dimensiones correctas, lo siguiente fue la creación de la imagen en formato pbm, esto se logró mediante una biblioteca por parte de openCV y creando una función para esto.

```
def Imagen(matriz, NombreArchivo):
    cv2.imwrite(NombreArchivo, matriz)
Imagen(ImagenSalida, "imagen.pbm")
```

Selección de n vecinos dado Knn

Uno de los requerimientos para solucionar el problema era la cantidad de vecinos más cercanos para la toma de decisión en la clasificación, ejemplo:



El valor de K era crucial para la clasificación, dado que gracias a este, podemos definir la asignación de la etiqueta a predecir, pero como vemos en el gráfico, modificar el valor de K puede afectar el resultado final, por esto realizamos pruebas con números primos (dado que así prevenimos un posible empate) que en este caso son, 3, 5, 7, 51 y 149, el procedimiento fue ejecutado 30 veces con cada valor y obtenemos un promedio de su afinidad/asertividad con valores de 0 a 1 (donde 0 es muy bajo y 1 es total).

El resultado del procesamiento con diferentes valores de K es:

Al evaluar diferentes valores de K en el algoritmo identificamos que K = 3 ofrece la mejor precisión. Analizamos los resultados de 30 ejecuciones del código y encontramos que, en promedio, K = 3 supera a otros valores como:
K = 5, K = 7, K = 51 y K = 149.

Elegir K = 3 proporciona un resultado aceptable. Es lo suficientemente grande para capturar patrones locales sin ser excesivamente influenciado por el ruido (errores posibles de otros procesamientos), lo que puede ocurrir con K = 1. También evita las ambigüedades excesivas que vemos con valores altos como K = 51 y K = 149, donde el modelo pierde detalle y precisión.

La estabilidad es otro punto fuerte. Con K = 3, el modelo no solo es preciso, sino que también muestra resultados consistentes en múltiples ejecuciones, lo que refuerza su fiabilidad. En resumen, los datos respaldan claramente que K = 3 es

la mejor opción para nuestro problema de clasificación, combinando alta precisión con estabilidad y una buena resistencia al ruido.
En nuestro caso la diferencia entre el máximo y el mínimo alcanzado se encuentra entre 92 y 100 (0.92 y 1) pero con un promedio de 96.93 con una variación de +-0.34.

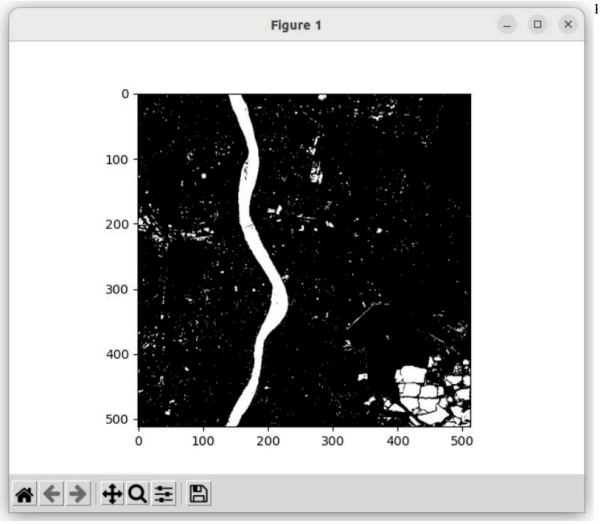
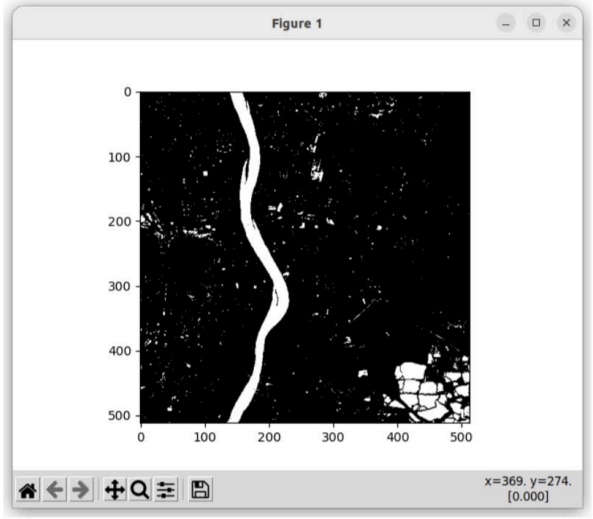
Pruebas de cada KNN

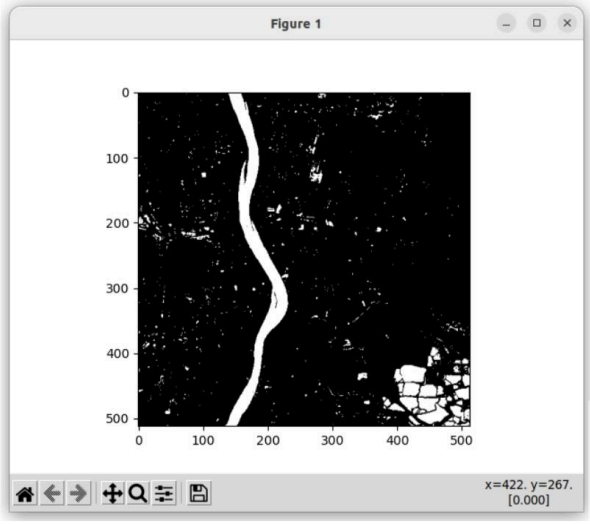
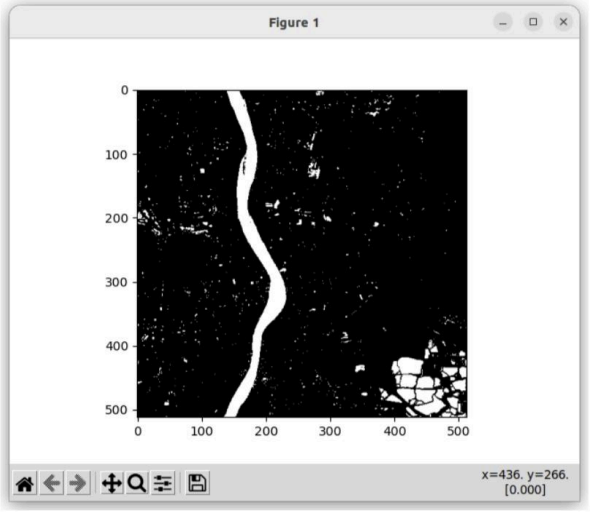
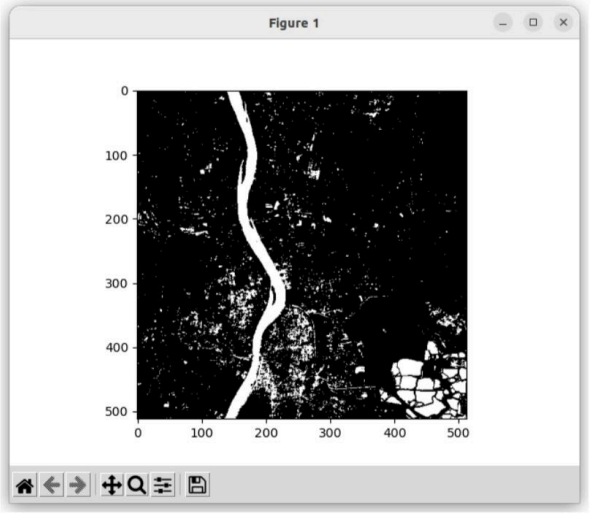
En la siguiente tabla se puede observar el porcentaje de asertividad de cada uno de los k vecinos puestos a prueba por nosotros, cumpliendo con los k vecinos propuestos por el mismo proyecto y añadiendo uno extra para confirmar que entre más vecinos se seleccionen para la clasificación es menos el porcentaje de asertividad que tiene este, dado que entre menos vecinos, se puede alcanzar incluso el 100 por ciento de asertividad y entre más vecinos haya (149 máximo) este alcanza un máximo de 78 por ciento de asertividad.

Tabla representando los porcentajes de asertividad

K3	K5	K7	K51	K149
98	96	96	100	62
98	96	96	94	46
98	98	100	96	38
98	98	98	94	48
96	96	96	98	42
100	98	94	98	46
98	98	96	96	44
98	98	94	100	42
92	94	94	96	48
94	98	98	90	62
96	92	98	96	44
100	96	90	88	58
96	98	100	88	58
96	98	96	86	58
96	98	96	94	58
98	94	98	100	58
98	94	96	92	78
96	98	98	96	40
100	100	96	94	48
96	98	98	98	40
98	100	96	96	48
98	94	98	100	48
94	100	100	92	74
94	98	96	92	46
98	98	100	90	40
96	94	94	98	46
96	96	98	96	48
96	96	100	100	42
98	94	98	96	42
98	96	96	94	44

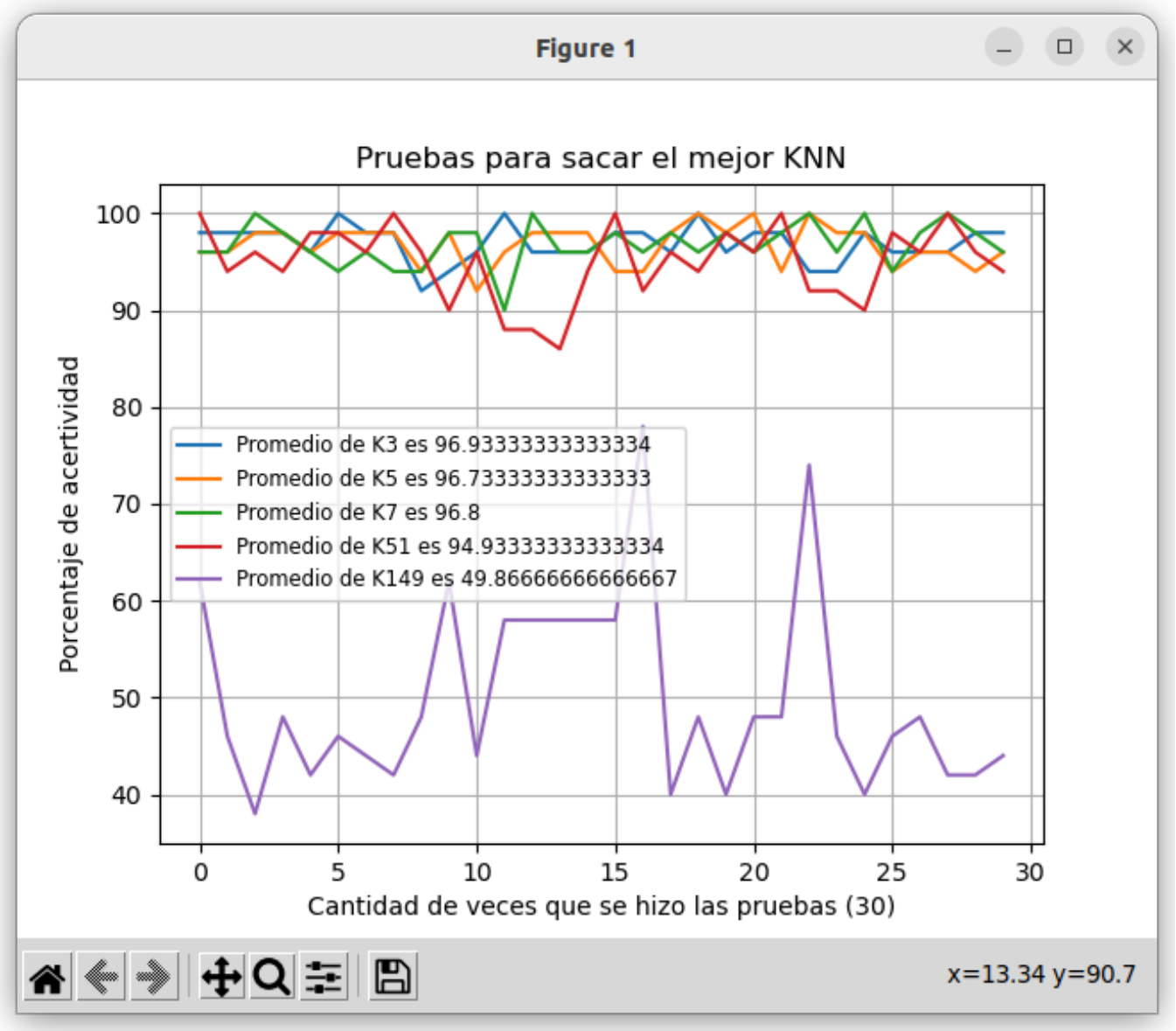
Mapeo dado diferentes KNN.

Imagen resultante con diferentes KNN	**K vecino más cercano con su porcentaje de asertividad**
	Número de K vecinos = 3 Para este caso se utilizaron 3 KNN con lo que dio un promedio de porcentaje de asertividad (dado 30 ejecuciones con la misma cantidad de vecinos) del **96.93** y generando la imagen de la izquierda.
K5 	Número de K vecinos = 5 Para este caso se utilizaron 5 KNN con lo que dio un promedio de porcentaje de asertividad (dado 30 ejecuciones con la misma cantidad de vecinos) del **96.73** y generando la imagen de la izquierda.

K7  	Número de K vecinos = 7 Para este caso se utilizaron 7 KNN con lo que dio un promedio de porcentaje de asertividad (dado 30 ejecuciones con la misma cantidad de vecinos) del **96.80** y generando la imagen de la izquierda.
K51  	Número de K vecinos = 51 Para este caso se utilizaron 51 KNN con lo que dio un promedio de porcentaje de asertividad (dado 30 ejecuciones con la misma cantidad de vecinos) del **94.93** y generando la imagen de la izquierda.
K149  	Número de K vecinos = 149 Para este caso se utilizaron 149 KNN con lo que dio un promedio de porcentaje de asertividad (dado 30 ejecuciones con la misma cantidad de vecinos) del **49.86** y generando la imagen de la izquierda.

Muestra de los porcentajes obtenidos por cada KNN y la representación gráfica de estos mismos.

En la imagen podemos observar que el mejor porcentaje de asertividad fue el K3, superando eventualmente al resto con un promedio de 96.93.



Conclusiones

El desarrollo de este programa resultó particularmente interesante debido a los conocimientos teóricos adquiridos en clase. La aplicación de las funciones y metodologías teóricas aprendidas siempre supone un desafío al intentar trasladarlas al ámbito práctico. Sin embargo, la elección del lenguaje de programación fue un proceso sencillo, ya que contábamos con experiencia previa en Python. Además, la sintaxis de las funciones que utilizamos resultó ser bastante accesible y sencillo.

En cuanto a la implementación de la lógica del programa, descubrimos que no fue tan complicada como habíamos anticipado. Esto se debió, en gran medida, al apoyo que encontramos en diversas bibliotecas que ofrecían numerosas herramientas útiles para el procesamiento de datos. Estas bibliotecas facilitaron enormemente nuestra tarea, permitiéndonos enfocarnos más en la estructura y funcionamiento del programa, en lugar de preocuparnos por aspectos básicos de codificación.

La combinación de nuestros conocimientos teóricos, la familiaridad con el lenguaje Python y el uso estratégico de bibliotecas especializadas nos permitió desarrollar un programa de manera efectiva y menos complicada de lo previsto inicialmente. Este proceso no solo reforzó nuestros conocimientos, sino que también nos brindó una valiosa experiencia práctica en el procesamiento de datos.

Bibliografía y Consultas

https://python-para-impacientes.blogspot.com/2020/01/leer-y-escribir-arrays-numpy-en-archivos.html